

گزارش پروژه نرم افزار ریاضی

امیر محمد ذاکر

فهرست مطالب

۲	۱	درونیابی لاگرانژ و اسپلاین مکعبی
۲	۱.۱	مراحل و نحوه پیاده‌سازی الگوریتم‌ها
۲	۲.۱	کد پایتون پیاده‌سازی شده
۳	۳.۱	خروجی دقیق برنامه در محیط ترمینال
۴	۴.۱	ضابطه چندجمله‌ای درونیاب لاگرانژ
۴	۵.۱	ضابطه تکه‌ای درونیاب اسپلاین مکعبی
۴	۶.۱	نمودار مقایسه‌ای و تحلیل کامل رفتار توابع
۶	۲	تحلیل آماری و تصویرسازی داده‌های زمان اجرای الگوریتم‌ها
۶	۱.۲	مراحل و نحوه پیاده‌سازی الگوریتم‌ها
۶	۲.۲	کد پایتون پیاده‌سازی شده
۷	۳.۲	خروجی دقیق برنامه در محیط ترمینال
۸	۴.۲	تحلیل آماری میانگین زمان اجرا
۸	۵.۲	نمودارهای ترسیم شده و تحلیل جامع رفتار الگوریتم‌ها
۱۰	۱.۵.۲	تحلیل نمودار خطی و بررسی روند رشد
۱۰	۲.۵.۲	تحلیل نمودار میله‌ای و مقایسه نقطه‌ای عملکرد
۱۰	۳.۵.۲	تحلیل نمودار جعبه‌ای و بررسی ثبات توزیع
۱۱	۳	انتگرال‌گیری عددی به روش‌های دوزنقه‌ای، سیمپسون و کوادراتور گوسی
۱۱	۱.۳	مراحل و نحوه پیاده‌سازی الگوریتم‌ها
۱۱	۲.۳	کد پایتون پیاده‌سازی شده
۱۲	۳.۳	خروجی دقیق برنامه در محیط ترمینال
۱۲	۴.۳	تحلیل و مقایسه جامع نتایج عددی روش‌ها
۱۲	۱.۴.۳	روش دوزنقه‌ای
۱۳	۲.۴.۳	روش سیمپسون
۱۳	۳.۴.۳	روش کوادراتور گوسی
۱۳	۴.۴.۳	نتیجه‌گیری مقایسه‌ای
۱۴	۴	پیوست: لینک گیت‌هاب

۱ درونیابی لاگرانژ و اسپلاین مکعبی

در این بخش، هدف یافتن توابع درونیاب برای نقاط داده شده در صورت پروژه است. نقاط مشخص شده عبارتند از: $(1, 1)$ ، $(2, 3)$ ، $(3, 5)$ ، $(4, 8)$ ، $(5, 5)$ و $(6, 2)$. برای این داده‌ها، دو روش درونیابی لاگرانژ^۱ و اسپلاین مکعبی طبیعی^۲ در محیط پایتون^۳ پیاده‌سازی شده است.

۱.۱ مراحل و نحوه پیاده‌سازی الگوریتم‌ها

پروژه پایتون مربوط به این بخش شامل گام‌های زیر است:

۱. تعریف و مقداردهی اولیه نقاط: ابتدا نقاط به صورت دو آرایه مجزا برای مولفه‌های x و y در کتابخانه نام‌پای^۴ تعریف می‌شوند.

۲. درونیابی لاگرانژ: برای درونیابی لاگرانژ و جلوگیری از ناپایداری‌های عددی معمول، از کلاس پیشرفته BarycentricInterpolator در کتابخانه سای‌پای^۵ استفاده شده است.

۳. استخراج محاسباتی ضابطه ریاضی: به دلیل اینکه کتابخانه‌های محاسبات عددی صرفاً خروجی عددی می‌دهند، با استفاده از کتابخانه سیم‌پای^۶ فرمول نمادین پایه‌های لاگرانژ پیاده‌سازی و در نهایت با دستور expand ساده‌سازی شده است تا ضابطه دقیق باز شود.

۴. درونیابی اسپلاین مکعبی: با استفاده از کلاس CubicSpline و با تنظیم پارامتر `bc_type='natural'` شرط مرزی صفر بودن مشتق دوم در نقاط ابتدایی و انتهایی اعمال شده و ضرایب توابع تکه‌ای استخراج گردیده‌اند.

۵. تصویرسازی و ذخیره‌سازی: با کمک کتابخانه مت‌پلات‌لیب^۷ هر دو نمودار به همراه نقاط اصلی رسم و با کیفیت بالا ذخیره می‌شوند.

۲.۱ کد پایتون پیاده‌سازی شده

کد کامل پایتون که برای انجام محاسبات این بخش نوشته شده، به شرح زیر است:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.interpolate import BarycentricInterpolator,
4   CubicSpline
5 from sympy import symbols, expand
6
7 # 1. Define the data points given in the project prompt
8 x_data = np.array([1, 2, 3, 4, 5, 6])
9 y_data = np.array([1, 3, 5, 8, 5, 2])
10
11 print("--- Part 1: Lagrange Interpolation ---")
12 # 2. Perform Lagrange interpolation using BarycentricInterpolator
13 lagrange_interpolator = BarycentricInterpolator(x_data, y_data)
14
15 # Extract the exact mathematical formula using sympy for clear
16   display
```

Lagrange Interpolation^۱
Natural Cubic Spline^۲
Python^۳
NumPy^۴
SciPy^۵
SymPy^۶
Matplotlib^۷

```

15 x_sym = symbols('x')
16 lagrange_expr = 0
17 for i in range(len(x_data)):
18     # Construct Lagrange basis polynomials
19     p = 1
20     for j in range(len(x_data)):
21         if i != j:
22             p *= (x_sym - x_data[j]) / (x_data[i] - x_data[j])
23     lagrange_expr += y_data[i] * p
24
25 lagrange_simplified = expand(lagrange_expr)
26 print("Exact formula for Lagrange interpolating polynomial:")
27 print(f"L(x) = {lagrange_simplified}\n")
28
29 print("--- Part 2: Spline Interpolation ---")
30 # 3. Perform Cubic Spline interpolation (Natural type)
31 cs = CubicSpline(x_data, y_data, bc_type='natural')
32
33 print("Exact piecewise formulas for the Cubic Spline (per
    interval):")
34 for i in range(len(x_data) - 1):
35     coeffs = cs.c[:, i]
36     d, c, b, a = coeffs[0], coeffs[1], coeffs[2], coeffs[3]
37     print(f"Interval [{x_data[i]}, {x_data[i+1]}]:")
38     print(f"    S_{i}(x) = {a} + {b:.4f}(x - {x_data[i]}) + {c:.4f}
        (x - {x_data[i]})^2 + {d:.4f}(x - {x_data[i]})^3")
39
40 # 4. Plotting the results for the final report
41 x_plot = np.linspace(1, 6, 500)
42 y_lagrange_plot = lagrange_interpolator(x_plot)
43 y_spline_plot = cs(x_plot)
44
45 plt.figure(figsize=(10, 6))
46 plt.scatter(x_data, y_data, color='red', zorder=5, label='
    Original Data Points')
47 plt.plot(x_plot, y_lagrange_plot, label='Lagrange Interpolation',
    linestyle='--', color='blue')
48 plt.plot(x_plot, y_spline_plot, label='Natural Cubic Spline',
    linestyle='-', color='green')
49 plt.title('Comparison of Lagrange and Cubic Spline Interpolation'
    )
50 plt.xlabel('X')
51 plt.ylabel('Y')
52 plt.grid(True, linestyle=':', alpha=0.6)
53 plt.legend()
54 plt.savefig('interpolation_plot.png', dpi=300)

```

۳.۱ خروجی دقیق برنامه در محیط ترمینال

پس از اجرای کد فوق، خروجی‌های دقیق ریاضی به صورت متنی در ترمینال به شرح زیر نمایش داده می‌شوند:

--- Part 1: Lagrange Interpolation ---

Exact formula for Lagrange interpolating polynomial:

$$L(x) = 7x^5/40 - 71x^4/24 + 147x^3/8 - 1249x^2/24 + 1369x/20 - 31$$

--- Part 2: Spline Interpolation ---

Exact piecewise formulas for the Cubic Spline (per interval):

Interval [1, 2]:

$$S_0(x) = 1.0 + 2.1866(x - 1) + 0.0000(x - 1)^2 + -0.1866(x - 1)^3$$

Interval [2, 3]:

$$S_1(x) = 3.0 + 1.6268(x - 2) + -0.5598(x - 2)^2 + 0.9330(x - 2)^3$$

Interval [3, 4]:

$$S_2(x) = 5.0 + 3.3062(x - 3) + 2.2392(x - 3)^2 + -2.5455(x - 3)^3$$

Interval [4, 5]:

$$S_3(x) = 8.0 + 0.1483(x - 4) + -5.3971(x - 4)^2 + 2.2488(x - 4)^3$$

Interval [5, 6]:

$$S_4(x) = 5.0 + -3.8995(x - 5) + 1.3493(x - 5)^2 + -0.4498(x - 5)^3$$

۴.۱ ضابطه چندجمله‌ای درونیاب لاگرانژ

با توجه به خروجی سیستم، ضابطه دقیق ریاضی ساده‌شده‌ی چندجمله‌ای لاگرانژ به صورت زیر است:

$$L(x) = \frac{7}{40}x^5 - \frac{71}{24}x^4 + \frac{147}{8}x^3 - \frac{1249}{24}x^2 + \frac{1369}{20}x - 31.$$

۵.۱ ضابطه تکه‌ای درونیاب اسپلاین مکعبی

فرمول ریاضی تکه‌ای توابع درجه سوم برازش‌شده بر روی فواصل مجاور به شرح زیر است:

$$\begin{aligned} S_0(x) &= 1.0 + 2.1866(x - 1) - 0.1866(x - 1)^3, & x \in [1, 2], \\ S_1(x) &= 3.0 + 1.6268(x - 2) - 0.5598(x - 2)^2 + 0.9330(x - 2)^3, & x \in [2, 3], \\ S_2(x) &= 5.0 + 3.3062(x - 3) + 2.2392(x - 3)^2 - 2.5455(x - 3)^3, & x \in [3, 4], \\ S_3(x) &= 8.0 + 0.1483(x - 4) - 5.3971(x - 4)^2 + 2.2488(x - 4)^3, & x \in [4, 5], \\ S_4(x) &= 5.0 - 3.8995(x - 5) + 1.3493(x - 5)^2 - 0.4498(x - 5)^3, & x \in [5, 6]. \end{aligned}$$

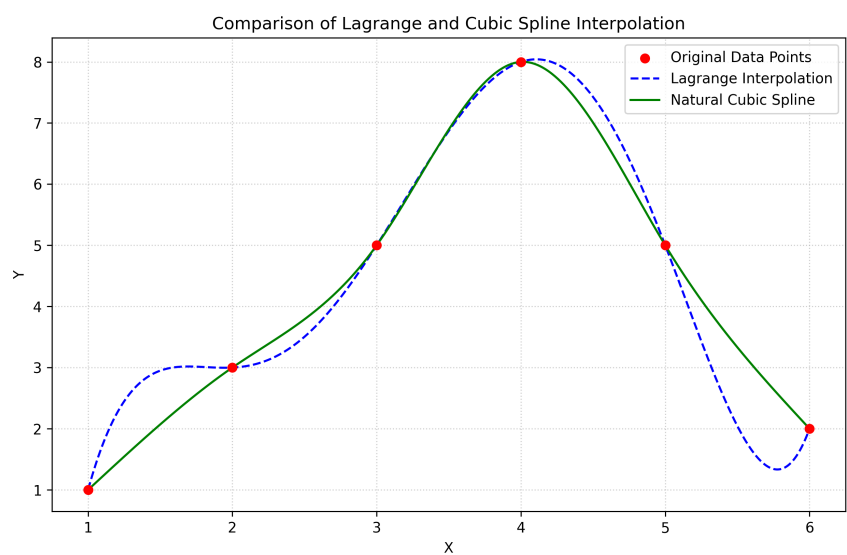
۶.۱ نمودار مقایسه‌ای و تحلیل رفتار توابع

شکل ۱ نشان‌دهنده نمودار خروجی حاصل از اجرای کد پایتون است.

تحلیل ریاضی رفتار این دو منحنی نشان می‌دهد که هر دو روش با موفقیت کامل از تمامی ۶ نقطه داده‌شده عبور کرده‌اند؛ اما تفاوت رفتار نوسانی عظیمی با یکدیگر دارند. چندجمله‌ای لاگرانژ به دلیل برخورداری از درجه بالا ($n - 1 = 5$), در بازه‌های ابتدایی و انتهایی دچار نوسانات شدید خارج از محدوده داده‌ها شده است. این رفتار نوسانی نامطلوب به پدیده رانگ^۸ معروف است که نشان می‌دهد توابع چندجمله‌ای درجه بالا برای نقاط با فواصل مساوی گزینه‌ی مناسبی نیستند.

در مقابل، روش اسپلاین مکعبی به دلیل ماهیت تکه‌ای بودن، در هر بازه تنها از توابع درجه ۳ استفاده می‌کند. برقراری شروط پیوستگی هم‌زمان تابع، مشتق اول و مشتق دوم در نقاط گره‌ای در کنار شرط مرزی طبیعی (صفر بودن مشتق دوم در پایانه‌ها)، سبب شده است تا منحنی اسپلاین رفتاری بسیار پایدار، نرم، عاری از هرگونه نوسان کاذب و کاملاً منطبق بر روند توزیع داده‌ها ارائه دهد.

Runge's phenomenon^۸



شکل ۱: نمودار مقایسه عملکرد درونیابی لاگرانژ و اسپلاین مکعبی طبیعی بر روی نقاط داده شده.

۲ تحلیل آماری و تصویرسازی داده‌های زمان اجرای الگوریتم‌ها

در این بخش، هدف بررسی هوشمند و پویای داده‌های مربوط به زمان اجرای سه الگوریتم مختلف بر حسب اندازه داده‌های ورودی از محدوده 100 الی 600 کیلوبایت و تحلیل تغییرات رفتار آن‌ها پس از افزودن سطر جدید 700 کیلوبایت است. تمامی این مراحل بدون هاردکد کردن داده‌ها و از طریق توابع پانداس^۹ انجام شده است.

۱.۲ مراحل و نحوه پیاده‌سازی الگوریتم‌ها

مراحل پیاده‌سازی این بخش در زبان پایتون^{۱۰} به شرح زیر است:

۱. فراخوانی پویا: با استفاده از تابع `read_excel`، فایل داده‌ها مستقیماً خوانده شده و با متد `strip` نام ستون‌ها پاک‌سازی می‌شوند تا فواصل خالی احتمالی باعث ایجاد خطا در محاسبات نگردند.

۲. محاسبه شاخص‌های آماری: پیش از اعمال تغییرات در ساختار فایل، میانگین ستون مربوط به الگوریتم ۲ محاسبه و ثبت می‌شود.

۳. تزریق پویای داده‌ی جدید: ردیف جدید مربوط به داده‌ی 700 کیلوبایت به کمک تابع `concat` به انتهای ماتریس داده‌ها افزوده می‌شود تا در اجرای مجدد، نمودارها به صورت خودکار به‌روزرسانی شوند.

۴. تولید نمودارها: با بهره‌گیری از کتابخانه‌های مت‌پلات‌لیب^{۱۱} و سی‌بورن^{۱۲}، سه نمودار خطی، میله‌ای و جعبه‌ای برای تحلیل همه‌جانبه توزیع و روند داده‌ها رسم می‌شوند.

۲.۲ کد پایتون پیاده‌سازی شده

کد کامل پایتون توسعه‌یافته برای این بخش به صورت زیر است:

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 # Set style for professional-looking plots
6 sns.set_theme(style="whitegrid")
7
8 # 1. Reading the Excel file dynamically
9 excel_file = '3.xlsx'
10 df = pd.read_excel(excel_file)
11 df.columns = df.columns.strip()
12
13 # 2. Calculating the mean for Alg.2 (Before adding the new data row)
14 mean_alg2 = df['Alg.2'].mean()
15 print("--- Part 2 (c): Statistical Analysis ---")
16 print(f"Mean execution time for Alg.2 (100KB to 600KB): {mean_alg2:.2f}\n")
17
18 # 3. Dynamically appending new data row (700KB row)
19 new_row = {
20     'Run time for different data size': '700KB',
21     'Alg.1': 80,
22     'Alg.2': 320,
```

Pandas^۹
Python^{۱۰}
Matplotlib^{۱۱}
Seaborn^{۱۲}

```

23     'Alg.3': 700
24 }
25 df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)
26 print("--- Part 2 (b): Updated Dataframe with 700KB Row ---")
27 print(df)
28 print("\n")
29
30 size_col = df.columns[0]
31 algorithms = [col for col in df.columns if col != size_col]
32
33 # 4. Plotting Bar, Line, and Box Plots
34 # Line Plot
35 plt.figure(figsize=(10, 6))
36 for alg in algorithms:
37     plt.plot(df[size_col], df[alg], marker='o', linewidth=2, label=
38         alg)
39 plt.title('Execution Time Trend across Different Data Sizes',
40     fontsize=14, fontweight='bold')
41 plt.xlabel('Data Size', fontsize=12)
42 plt.ylabel('Execution Time (ms)', fontsize=12)
43 plt.legend()
44 plt.tight_layout()
45 plt.savefig('execution_time_line_plot.png', dpi=300)
46
47 # Bar Plot
48 df_long = pd.melt(df, id_vars=[size_col], value_vars=algorithms,
49     var_name='Algorithm', value_name='Execution Time')
50 plt.figure(figsize=(10, 6))
51 sns.barplot(data=df_long, x=size_col, y='Execution Time', hue='
52     Algorithm', palette='muted')
53 plt.title('Comparison of Algorithm Execution Times per Data Size',
54     fontsize=14, fontweight='bold')
55 plt.xlabel('Data Size', fontsize=12)
56 plt.ylabel('Execution Time (ms)', fontsize=12)
57 plt.tight_layout()
58 plt.savefig('execution_time_bar_plot.png', dpi=300)
59
60 # Box Plot
61 plt.figure(figsize=(8, 6))
62 sns.boxplot(data=df[algorithms], palette='pastel')
63 plt.title('Distribution of Execution Times per Algorithm', fontsize
64     =14, fontweight='bold')
65 plt.xlabel('Algorithm', fontsize=12)
66 plt.ylabel('Execution Time Range', fontsize=12)
67 plt.tight_layout()
68 plt.savefig('execution_time_box_plot.png', dpi=300)

```

۳.۲ خروجی دقیق برنامه در محیط ترمینال

متن خروجی حاصل از محاسبات و چاپ ماتریس داده‌ها به شرح زیر است:

--- Part 2 (c): Statistical Analysis ---

Mean execution time for Alg.2 (100KB to 600KB): 250.00

--- Part 2 (b): Updated Dataframe with 700KB Row ---

Run time for different data size		Alg.1	Alg.2	Alg.3
0	100KB	50	200	100
1	200KB	55	220	200
2	300KB	60	240	300
3	400KB	65	260	400
4	500KB	70	280	500
5	600KB	75	300	600
6	700KB	80	320	700

۴.۲ تحلیل آماری میانگین زمان اجرا

مطابق با خواسته بند «ج» پروژه، میانگین زمان اجرای مربوط به الگوریتم ۲ برای اندازه‌های اولیه داده‌ها (100 تا 600 کیلوبایت) بر اساس فرمول زیر محاسبه شده است:

$$\mu = \frac{1}{N} \sum_{i=1}^N X_i = \frac{200 + 220 + 240 + 260 + 280 + 300}{6} = 250.00 \text{ ms}.$$

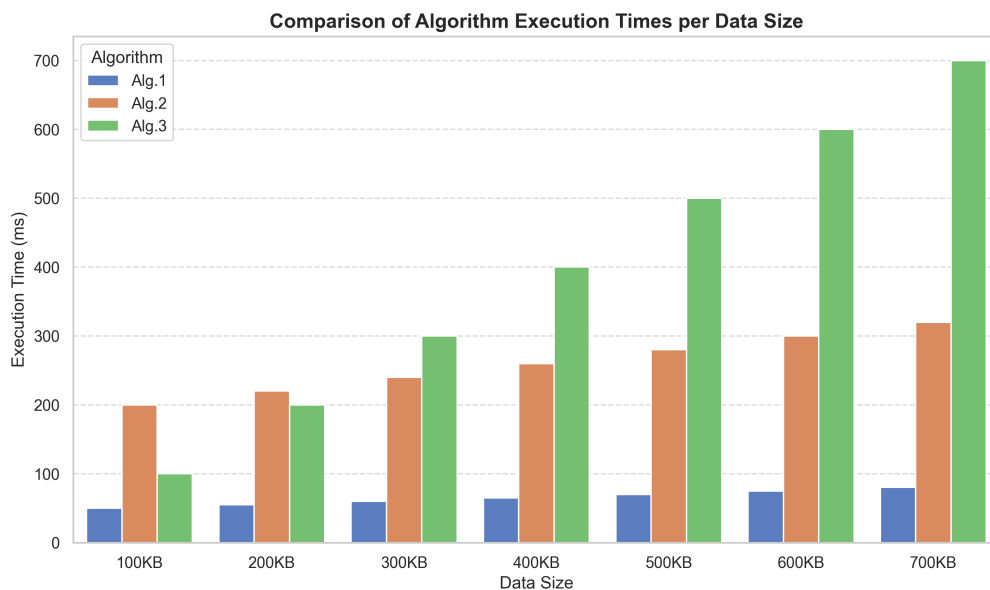
این شاخص آماری نشان می‌دهد که رفتار کلی الگوریتم ۲ در بازه داده‌های ورودی استاندارد، حول مرکز ثقل 250 میلی‌ثانیه نوسان می‌کند.

۵.۲ نمودارهای ترسیم شده و تحلیل جامع رفتار الگوریتم‌ها

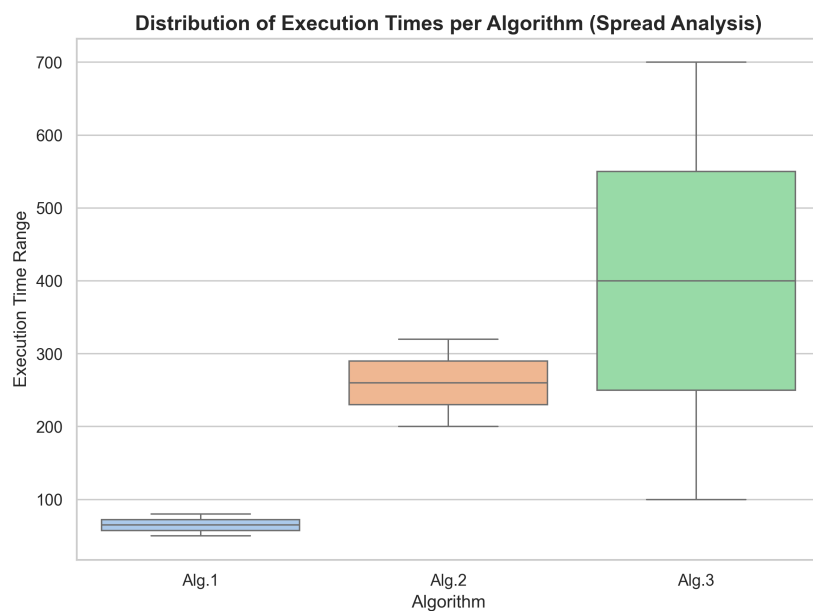
نمودارهای زیر خروجی‌های تصویری به‌دست‌آمده از کد پایتون هستند که پس از تزریق خودکار داده‌ی سطر 700 کیلوبایت ترسیم شده‌اند.



شکل ۲: روند تغییرات زمان اجرای الگوریتم‌ها بر حسب افزایش اندازه ورودی.



شکل ۳: مقایسه ستونی و چندگانه زمان اجرای الگوریتم‌ها در هر یک از سطوح اندازه داده.



شکل ۴: نمودار جعبه‌ای تحلیل دامنه توزیع، میانه و پراکندگی کلی زمان اجرای الگوریتم‌ها.

۱.۵.۲ تحلیل نمودار خطی و بررسی روند رشد

با توجه به شکل ۲، رفتار رشد زمان اجرای هر سه الگوریتم به وضوح قابل تحلیل است:

- الگوریتم ۱: این الگوریتم با شیبی بسیار ملایم رشد می‌کند و در تمام طول آزمایش کمترین زمان اجرا را به خود اختصاص داده است؛ در نتیجه، بهینه‌ترین الگوریتم از نظر پیچیدگی زمانی^{۱۳} است.
- الگوریتم ۲: روندی کاملاً خطی و با شیب ثابت دارد. اگرچه زمان شروع آن از الگوریتم ۳ در حجم‌های کوچک بیشتر است، اما نرخ رشد کنترل‌شده‌ای دارد.
- الگوریتم ۳: این الگوریتم دارای تندترین شیب صعودی است. زمان اجرای آن با افزایش اندازه ورودی به شدت بالا می‌رود و حالت ناپایداری در حجم‌های بزرگ پیدا می‌کند.

۲.۵.۲ تحلیل نمودار میله‌ای و مقایسه نقطه‌ای عملکرد

بر اساس شکل ۳، در اندازه‌های ورودی کوچک مانند 100 و 200 کیلوبایت، الگوریتم ۳ سریع‌تر از الگوریتم ۲ عمل می‌کند. اما یک نقطه تلاقی مهم در محدوده 300 کیلوبایت رخ می‌دهد. از حجم 400 کیلوبایت به بعد، الگوریتم ۲ گوی سبقت را از الگوریتم ۳ می‌رباید؛ چرا که زمان اجرای الگوریتم ۳ به شکل تصاعدی افزایش یافته و در حجم 700 کیلوبایت به اوج خود یعنی 700 میلی‌ثانیه می‌رسد. این امر عدم مقایسه‌پذیری^{۱۴} الگوریتم ۳ را اثبات می‌کند.

۳.۵.۲ تحلیل نمودار جعبه‌ای و بررسی ثبات توزیع

نمودار جعبه‌ای در شکل ۴، دامنه تغییرات داده‌ها را به تصویر می‌کشد:

- جعبه مربوط به الگوریتم ۱۵۱ بسیار فشرده و در پایین‌ترین سطح قرار دارد که نشان‌دهنده پایداری بالا، انحراف معیار ناچیز و واریانس بسیار کم در شرایط کاری مختلف است.
- جعبه مربوط به الگوریتم ۱۶۳ بیشترین ارتفاع (طول بازه مابین چارک اول و سوم) را دارد. این گستردگی نشان‌دهنده حساسیت فوق‌العاده بالا و عدم ثبات زمان اجرای این الگوریتم نسبت به تغییر ابعاد ورودی سیستم است.

^{۱۳} Time Complexity

^{۱۴} Scalability

^{۱۵} Alg.1

^{۱۶} Alg.3

۳ انتگرال گیری عددی به روش های دوزنقه ای، سیمپسون و کوادراتور گوسی

در این بخش، هدف محاسبه عددی مقدار انتگرال تابع مشخص شده در صورت پروژه یعنی $f(x) = e^{x^2}$ در بازه کران دار $(0, 1)$ است. از آنجا که این تابع فاقد پادمشتق بر حسب توابع مقدماتی است، مقدار انتگرال آن صرفاً از طریق روش های تقریبی عددی محاسبه می شود. سه روش دوزنقه ای^{۱۷}، سیمپسون^{۱۸} و کوادراتور گوسی^{۱۹} در محیط پایتون^{۲۰} روی این تابع اعمال شده اند.

۱.۳ مراحل و نحوه پیاده سازی الگوریتم ها

مراحل محاسباتی و الگوریتمی این بخش به شرح زیر است:

۱. تعریف تابع هدف: ابتدا تابع ریاضی $f(x) = e^{x^2}$ با بهره گیری از تابع نمایی آرایه نام پای^{۲۱} یعنی `exp.np` پیاده سازی می شود.

۲. شبکه بندی بازه انتگرال گیری: برای اعمال روش های مبتنی بر نیوتن-کوتس^{۲۲} (دوزنقه ای و سیمپسون)، بازه کل $[0, 1]$ به کمک تابع `linspace.np` به تعداد 100 زیربازه هم اندازه با مجموع 101 نقطه گره ای تقسیم بندی می شود.

۳. اعمال قاعده دوزنقه ای و سیمپسون: با استفاده از توابع اختصاصی و بهینه `trapezoid` و `simpson` از زیربسته محاسبات انتگرال سای پای^{۲۳} (`integrate.scipy`)، ارزیابی عددی با دریافت فواصل نقشه گره ای انجام می پذیرد.

۴. اعمال قاعده کوادراتور گوس-کرونرود: برای محاسبه انتگرال گوسی با بالاترین درجه دقت چندجمله ای، از تابع قدرتمند `quad` استفاده شده است که علاوه بر مقدار انتگرال، تخمین دقیقی از خطای مطلق سقف محاسبات را نیز در خروجی برمی گرداند.

۲.۳ کد پایتون پیاده سازی شده

کد کامل پایتون توسعه یافته برای محاسبات انتگرال گیری عددی به شرح زیر است:

```
1 import numpy as np
2 from scipy.integrate import trapezoid, simpson, quad
3
4 # 1. Define the target function f(x) = e^(x^2)
5 def f(x):
6     return np.exp(x**2)
7
8 # Define the integration intervals (from 0 to 1)
9 a = 0
10 b = 1
11
12 # Number of points for Trapezoidal and Simpson's methods
13 num_points = 101
14 x_intervals = np.linspace(a, b, num_points)
15 y_values = f(x_intervals)
16
17 print("--- Part 3 (a): Trapezoidal and Simpson's Methods ---")
18
19 # 2. Calculate using the Trapezoidal Rule
```

^{۱۷}Trapezoidal Rule

^{۱۸}Simpson's Rule

^{۱۹}Gaussian Quadrature

^{۲۰}Python

^{۲۱}NumPy

^{۲۲}Newton-Cotes formulas

^{۲۳}SciPy

```

20 integral_trapezoidal = trapezoid(y_values, x_intervals)
21 print(f"Trapezoidal Rule Result: {integral_trapezoidal:.8f}")
22
23 # 3. Calculate using Simpson's Rule
24 integral_simpson = simpson(y_values, x_intervals)
25 print(f"Simpson's Rule Result: {integral_simpson:.8f}\n")
26
27 print("--- Part 3 (b): Gaussian Quadrature Method ---")
28
29 # 4. Calculate using Gaussian Quadrature
30 integral_gaussian, abs_error = quad(f, a, b)
31 print(f"Gaussian Quadrature Result: {integral_gaussian:.8f}")
32 print(f"Estimated Absolute Error: {abs_error:.2e}\n")
33
34 print("--- Summary & Comparison ---")
35 print(f"Difference (Gaussian vs Simpson): {abs(integral_gaussian
36 - integral_simpson):.2e}")
37 print(f"Difference (Gaussian vs Trapezoidal): {abs(integral_gaussian
38 - integral_trapezoidal):.2e}")

```

۳.۳ خروجی دقیق برنامه در محیط ترمینال

مقادیر عددی خروجی چاپ شده در ترمینال پس از اجرای اسکریپت فوق به صورت زیر ثبت شده است:

--- Part 3 (a): Trapezoidal and Simpson's Methods ---

Trapezoidal Rule Result: 1.46269705

Simpson's Rule Result: 1.46265175

--- Part 3 (b): Gaussian Quadrature Method ---

Gaussian Quadrature Result: 1.46265175

Estimated Absolute Error: 1.62e-14

--- Summary & Comparison ---

Difference (Gaussian vs Simpson): 3.02e-09

Difference (Gaussian vs Trapezoidal): 4.53e-05

۴.۳ تحلیل و مقایسه جامع نتایج عددی روش‌ها

با توجه به نتایج حاصل از خروجی سیستم، مقدار تقریبی انتگرال برای هر سه روش با دقت اعشاری بالا استخراج گردید. فرمول ریاضی هر یک از روش‌ها و تحلیل خطای آن‌ها به شرح زیر تبیین می‌شود.

۱.۴.۳ روش دوزنقه‌ای

در این روش، تابع در هر زیربازه با یک خط راست تقریب زده می‌شود. فرمول محاسباتی آن به صورت زیر است:

$$I_T = \frac{h}{2} \left[f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n) \right].$$

مقدار حاصل از این روش برابر با 1.46269705 به دست آمده است. این روش به دلیل استفاده از تقریب‌های مرتبه اول خطی، دارای خطای قطع کلی از مرتبه $\mathcal{O}(h^2)$ است. تفاوت این روش با روش گوسی برابر با 4.53×10^{-5} است که بیشترین میزان انحراف را در میان روش‌های مورد آزمایش نشان می‌دهد.

۲.۴.۳ روش سیمپسون

در روش سیمپسون، تقریب توابع در هر دو زیربازه مجاور با استفاده از سهمی‌های درجه دوم انجام می‌گیرد. فرمول ریاضی این قاعده به صورت زیر است:

$$I_S = \frac{h}{3} \left[f(x_0) + 4 \sum_{i=1,3,\dots}^{n-1} f(x_i) + 2 \sum_{j=2,4,\dots}^{n-2} f(x_j) + f(x_n) \right].$$

مقدار عددی حاصل از محاسبات سیمپسون برابر با 1.46265175 است. خطای قطع این روش از مرتبه $\mathcal{O}(h^4)$ است. به دلیل انحنای سهمی توابع برازش شده، این روش انطباق بسیار بالایی با ساختار هموار تابع e^{x^2} دارد، به طوری که تفاوت خروجی آن با روش گوسی صرفاً برابر با 3.02×10^{-9} است که نشان‌دهنده دقت فوق‌العاده بالای آن در تعداد بازه داده‌شده است.

۳.۴.۳ روش کوادراتور گوسی

روش کوادراتور گوسی برخلاف دو روش قبل که از فواصل یکنواخت و ثابت استفاده می‌کردند، نقاط نمونه‌برداری x_i و وزن‌های متناظر w_i را به صورت بهینه و بر اساس ریشه‌های چندجمله‌ای‌های لژاندر^{۲۴} انتخاب می‌کند تا چندجمله‌ای‌های درجه بالا را دقیقاً انتگرال‌گیری کند:

$$I_G = \sum_{i=1}^m w_i f(x_i).$$

مقدار حاصل از این روش دقیق برابر با 1.46265175 با مرتبه خطای فوق‌العاده ناچیز و حد آستانه خطای مطلق محاسباتی معادل 1.62×10^{-14} به‌دست آمده است.

۴.۴.۳ نتیجه‌گیری مقایسه‌ای

با در نظر گرفتن روش کوادراتور گوسی به عنوان دقیق‌ترین معیار انتگرال‌گیری عددی (به دلیل ساختار متغیر و بهینه نقاط ارزیابی)، مقایسه شاخص‌ها اثبات می‌کند که روش سیمپسون با داشتن خطای ناچیز مرتبه نهم، به طرز چشمگیری نسبت به روش ذوزنقه‌ای (با خطای مرتبه پنجم) ارجحیت دارد. دلیل این امر، نرم بودن و مشتق‌پذیری بی‌نهایت تابع تحت انتگرال است که باعث می‌شود فرمول‌های مرتبه بالاتر مانند سیمپسون و کوادراتور گوسی با سرعت بسیار بالاتری به سمت مقدار واقعی انتگرال همگرا شوند.

^{۲۴}Legendre polynomials

۴ پیوست: لینک گیت‌هاب

<https://github.com/amirmohammadzaker/Mathematical-Software-Course>